

Applicability Filtering Model for S1000D documents using AND/OR trees

1st Theresia El Khoury
Nanomedicine Lab
Université Bourgogne Franche-Comté
Belfort, France
0000-0002-8383-4005
theresia.el-khoury@utbm.fr

2nd Georges Badr
Holy Spirit University of Kaslik
Jounieh, Lebanon
0000-0001-7404-9569
georgesbadr@usek.edu.lb

3rd Amir Hajjam El Hassani
Nanomedicine Lab
Université Bourgogne Franche-Comté
Belfort, France
0000-0002-8470-806X
amir.hajjam@utbm.fr

4th Stéphane N'Guyen Van Ky
Département de Recherche & Développement
STUDEC
Blagnac, France
0000-0003-3805-0305
sngvk@studec.fr

Abstract—Technical documentation written using the S1000D norm in aeronautics and aerospace fields is based on ‘applicability’ tags that define the context in which a document or part of it is valid. Filtering data modules by applicability is challenging because tags can have the potential to accumulate infinitely, reaching a level of complexity that becomes challenging for human interpretation. On the other hand, many changes were made to the applicability tag’s structure when upgraded from version 2.3 to version 4.1. This paper presents a general AND/OR tree-based model to filter documents based on their applicability. Transformations were made to both versions to get a unique one, thereby, evaluating the input values and selecting accessible content for each user. The model has shown accurate filtering for all tests made, on multiple datasets of different versions, having applicability of different complexity ranges.

Index Terms—AND/OR Trees, Data Filtering, Data Structure, Knowledge-based Filtering, S1000D

I. INTRODUCTION

It is very well known in all S1000D [1] based documents, that the applicability field is the most important, and complex to interpret. It specifies the criteria, such as system configurations, equipment variants, operational scenarios, or maintenance levels, under which the information is valid and accessible by users. After the vast change between versions 2.3 and 4.1 of the S1000D, the architecture of the applicability was remodeled, making its interpretation even more complex. In previous work, a S1000d AppLicability dEpended pRo-cessiNg mOdel - SALERNO [2] was created to convert the XML S1000D-based files, into a simpler format while preserving the different logic and relationships, including the applicability restrictions. As a second step, this study aims to decode both versions of applicability, and filter documents

by user’s criteria. It presents an And-Or tree-based model for applicability evaluation. Both versions were altered to a single, general syntax that can be used for both. The model was used to filter documents, as part of a knowledge-based filtering system. To the authors’ knowledge, no available articles tackle this subject in this manner.

The paper is structured as follows. The first section shows the related works of AND/OR tree uses. The structure of the applicability is described in the second section, including the difference between both versions and the changes made. The model’s architecture is defined in the second section, followed by the results in the fourth section. The conclusion summarizes the paper at the end.

II. RELATED WORKS

The idea of AND/OR trees was used in multiple domains. Authors in [3] introduce the effectiveness of a novel framework for constraint optimization, which is based on the concept of AND/OR search trees. The key advantage of utilizing this representation is its potential for a smaller size compared to traditional OR search trees. They presented a new generation of depth-first Branch-and-Bound algorithms that navigate through an AND/OR search space and employ the Mini-Bucket approximation scheme to generate heuristics for guiding the search process. The approach was not only competitive but also, in many instances, superior to state-of-the-art systematic search algorithms that explore the conventional OR space.

In a different domain, a ‘Game-Tree Search Using Proof Numbers’ was introduced in [4] based on the concept of AND/OR trees and their relevance to solving game positions or entire games. OR nodes represent positions where the first player plays, while AND nodes represent positions where the second player plays. A proof tree is a subtree of an AND/OR tree that contains a winning strategy for the first

This article is supported by STUDEC.

979-8-3503-6883-3/24/\$31.00 ©2024 IEEE

player, ensuring that a node is proven to be a win. The Proof-Number Search (PNS) algorithm is used, initializing the proof number (pn) and disproof number (dn) of the root. The search iteratively expands an AND/OR tree until the value of the root is decided - either pn or dn becomes 0, indicating proof or disproof.

To address the limitations of Artificial Intelligence algorithms used in web services, an AND/OR tree-based approach is adopted in [5] introducing the Input/Output Dependency AND/OR Tree representation. The research overcame the challenges and complexities associated with web service composition, offering a potentially more effective and efficient solution in the domain.

To our knowledge, no AND/OR model was created to represent applicability tags in XML-based documents written using S1000D norm.

III. APPLICABILITY STRUCTURE

The applicability field is used to filter and select the appropriate data modules, or parts of them, based on the specific requirements of the end-user or the targeted publication. By including this information, the documentation can be tailored to the specific configuration or operational context, providing the users with accurate and relevant information. It can be either a single condition or a combination of different and-or conditions, that can be infinitely embedded. In a general form, it is a combination of (Key, Value) pairs, where the key is the attribute, and the value represents one value, or range of values specific to this attribute.

A. Changes in versions

In version 2.3 below, the applicability tag had a child for each key, or attribute needed. In the later versions, all tags were replaced with only two:

- Assert: where attributes define the key and value;
- Evaluate: having different asset tags, with an attribute defining the AND/OR relationship between them.

Note that each 'evaluate' can also have another 'evaluate' child, leading to unlimited embedded tags. Table I shows the main difference between both versions tackled in this study.

These are the main points included in the transformation, all other tags in 2.3 are replaced by the 'assert' tag. The main target was to interpret the AND/OR logic behind the applicability.

1) *Transformation of both versions:* To have a single, general representation of applicability, appropriate changes for both versions were needed. The representation considers the main points of logic, with minimal complexity. All coding was written in Python. The applicability tags were extracted from each document and were transformed into JSON for an easier transformation. For version 2.3, the JSON dictionary is recursively traversed to convert the 'range' tags into the appropriate value as in version 4.1. The keys 'evaluate', 'assert', 'applicPropertyIdent', and 'applicPropertyValues' were added as well. The function considers the following:

TABLE I
REPRESENTATION OF LOGIC IN BOTH VERSIONS.

Logic	Version 2.3	Version 4.1
Key	Tag name	Value of the attribute 'applicPropertyIdent'
Value	Value of the key tag	Value of the attribute 'applicPropertyValues'
(Key, Value) pair	Single tag-value	Tag Assert having corresponding attributes
And	Implicitly shown by adding children tags	Explicitly defined by a value 'and' for 'andOr' attribute in evaluate
Or	Implicitly shown by adding same tags on the same level	Explicitly defined by a value 'or' for 'andOr' attribute in evaluate
Range of values	Tag 'range' with attributes 'from/to'	In the 'applicPropertyValues' the sign '~' is added between the values from/to

- If the value is a string, the (Key, Value) pair is converted to an assert, by adding the corresponding attributes;
- If the value is a list, the function is recursively called for each item and added to the dictionary having an 'evaluate' key of value 'or';
- If the value is a dictionary, the function is recursively called for each item and added to the dictionary having an 'evaluate' key of value 'and'.

The examples shown are taken from the public online S1000D reference. Fig. 1 shows the initial 2.3 JSON applicability. This is the result of the XML parsing, which will be converted in different steps. the first step, converting all keys having 'range' as a child. To represent a range of values, the sign '~' is added between the from/to values. To represent multiple values or a range of values, the sign '|' separates them. If only one range is set for a key, then the value is equal to the concatenated result of the 'from/to' values. In case multiple ranges are available, as in this example, then both ranges are converted and concatenated with the sign '|', to get the same interpreted value in version 4.1. Fig. 2 shows the result of this change in the case of multiple ranges.

The second step is to transform the JSON into the final structure, based on the 4.1 version, where only essential attributes are kept. Fig. 3 shows the result of the same applicability tag in both versions. The keys 'evaluate', 'andOr', 'assert', 'applicPropertyIdent', and 'applicPropertyValues' are added from version 2.3, and kept in 4.1. All others are removed from both versions.

IV. MODEL'S ARCHITECTURE

A general architecture was created to tackle both versions, based on the logical AND/OR trees. Each applicability dictionary is used to create its corresponding tree. Mainly composed of nodes classified by type as follows:

- 'AND' nodes, where all children should be evaluated as TRUE;
- 'OR' nodes, where at least one child should be TRUE;

```
{
  "model": {
    "@model": "Brook trekker",
    "version": {
      "@version": [
        {
          "@from": "001",
          "@to": "003"
        },
        {
          "@from": "051",
          "@to": "052"
        }
      ]
    }
  }
}
```

Fig. 1. Applicability example with multiple ranges.

- ‘LEAF’ nodes, having no children, and are compared with the input applicability values for evaluation of each tree.

A recursive function creates, for each JSON dictionary, a tree, with an ‘applic’ root of type ‘AND’, followed by AND/OR nodes, until creating all children for each node. The nodes of type ‘LEAF’ get their (Key, Value) pair from the values of the attributes of each assert; ‘applicPropertyIdent’ and ‘applicPropertyValues’. Some values can be predefined by each client, to represent ‘ALL’ values, meaning that no matter the value corresponding to this key, the evaluation is TRUE. If the value of a certain attribute is set to one of these, then any value is considered as TRUE. For this point, a list of values was created, and all pairs having the value in this list, are discarded in the node creation process. Thereby, the final tree of applicability is created by the AND/OR nodes, and leaves of (Key, Value) that are specific. So, for applicability

```
{
  "model": {
    "@model": "Brook trekker",
    "version": {
      "@range": "001~003|051~052"
    }
  }
}
```

Fig. 2. Range conversion of 2.3 applicability.

```
{
  "applic": {
    "evaluate": {
      "@andOr": "and",
      "assert": [
        {
          "@applicPropertyIdent": "model",
          "@applicPropertyValues": "Brook trekker"
        },
        {
          "@applicPropertyIdent": "version",
          "@applicPropertyValues": "001~003|051~052"
        }
      ]
    }
  }
}
```

Fig. 3. Final applicability structure.

having (1) model = ‘All’ and (2) version = ‘001~003’, all values of the attribute model, and only values between 001 and 003 inclusive for the attribute version are TRUE. The tree representing this applicability contains a node of type ‘AND’, with a ‘LEAF’ child having a ‘key’ set to version and a ‘value’ set to ‘001~003’.

A. Evaluating Trees

The process of filtering all applicability for a group of values relies on finding whether the given values are evaluated as TRUE or FALSE in each tree. The evaluation is also a recursive function, that takes as input the root of each tree, with a dictionary having the corresponding filtering values.

- If ‘AND’ node, recursively calls the function for all its children. All results should be TRUE to have a positive result;
- If ‘OR’ node recursively calls the function for all its children. Only one result should be TRUE to have a positive result;
- If ‘LEAF’ node then checks if the (Key, Value) pair is in the dictionary.

The function of similarity takes into consideration the following aspects for the ‘value’:

- The value in the node is split based on both characters ‘|’ for different values and ‘~’ for range value;
- The maintenance level of a user has hierarchical access. For a value set to ‘ml02’, users with a maintenance level equal to or greater than ‘ml02’ have access, while lower values don’t;
- If the value of the user is equal to, or in the range, then they’re similar, and FALSE in other cases.

V. EXAMPLE AND RESULTS

MongoDB database was used to store each segment of a document, and its corresponding applicability. These

documents were filtered by sections, based on each applicability given as input. This model underwent testing on various technical documents. Initially, it was evaluated using the online-accessible bike English dataset alongside the downloaded S1000D specifications [1]. Furthermore, it was applied to four additional datasets comprising both French and English versions, namely 4.1 and 2.3. Each dataset consisted of a varying number of documents ranging from 700 to 20,000, where different applicability was found, and for different sections of a document. Due to client confidentiality, these datasets are not publicly accessible. The outcomes of the evaluation demonstrated the model's successful performance. For all applicability that was initially extracted, all documents and segments of them were successfully filtered based on the given input values.

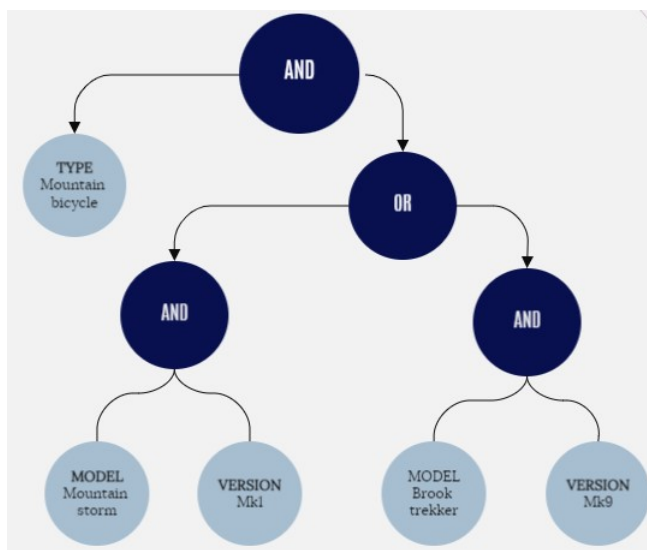


Fig. 4. Final applicability tree structure.

The example in Fig. 4 shows the tree result of both S1000D versions, having an AND/OR relationship. Based on a saved JSON, this tree is generated in real-time, to be evaluated based on the given (Key, Value) pairs as input. For this example, applicability is evaluated as TRUE, if it has a type of 'Mountain bicycle' with a model 'Mountain storm' of version 'MK1', or a type of 'Mountain bicycle' with a model 'Brook Trekker' of version 'MK9'. All other combinations of these three attributes are FALSE. As for applicability having other attributes, they are evaluated as TRUE, since they are not mentioned.

VI. CONCLUSION

This paper adapted an AND/OR tree-based model to filter S1000D documents based on their applicability. Two versions of the norm were addressed 2.3 and 4.1. The main idea was to have a general, common model representing both versions, knowing that the applicability structure changed from one version to another. Both versions were transformed into a

single format. For documents in version 2.3, all applicability tags were transformed into JSON and recursively traversed to convert the 'range' tags into the appropriate format value used in version 4.1. Additionally, in the same process, the (Key, Value) pairs were transformed to the 'assert' format found in 4.1, where the 'Key' is the value of the 'applicPropertyIdent' and the 'Value' for 'applicPropertyValues'. Dictionaries were converted into an 'evaluate' key, with the value 'AND', while lists to the value 'OR'.

The final JSON of both versions was saved in the MongoDB database and converted to an AND/OR tree for evaluation. the process of comparing the (Key, Value) pairs considered the range values, the hierarchical logic for certain values, and the case of keys accepting all values. Tests were made on different datasets, of different versions. For all cases that were taken into consideration, our model showed correct filtering, even for the complex context.

This model will be part of a hybrid search system for S1000D documentation. For each user's search, the first step is to filter all documents AND/OR document sections, that are visible and accessible to him.

REFERENCES

- [1] ASD, AIA, and A4A, "S1000d, international specification for technical publications using a common source database," n.d. [Online]. Available: <https://s1000d.org/>
- [2] T. El Khoury, G. Badr, A. H. El Hassani, and S. N. Van Ky, "S1000d applicability depended processing model-salerno," in *2023 14th International Conference on Information, Intelligence, Systems & Applications (IISA)*. IEEE, 2023, pp. 1–7.
- [3] R. Marinescu and R. Dechter, "And/or tree search for constraint optimization," in *Proc. of the 6th International Workshop on Preferences and Soft Constraints*, 2004, p. n.d.
- [4] A. Kishimoto, M. H. Winands, M. Müller, and J.-T. Saito, "Game-tree search using proof numbers: The first twenty years," *Icga Journal*, vol. 35, no. 3, pp. 131–156, 2012.
- [5] X. Ma, B. Dong, and M. He, "And/or tree search algorithm in web service composition," in *2008 IEEE Pacific-Asia Workshop on Computational Intelligence and Industrial Application*, vol. 2. IEEE, 2008, pp. 23–27.